

Codebook of UltraGrammer

ahbiee

Contents

1 你是否有...	1
1.1 嘗試...	1
1.2 檢查...	1
2 資料結構	1
2.1 BIT (Fenwick Tree)	1
2.2 Segment Tree 線段樹	2
3 數論: 尤拉公式, 數論分塊, Sprague-Grundy 函數, 排容原理	2
3.1 快速冪	2
3.2 模逆元與費馬小定理	2
3.3 質數檢查	2
3.4 大數乘法	2
4 字串	3
4.1 KMP 字串匹配	3
4.2 Z-Algorithm 字串匹配	3
4.3 Manacher 迴文判斷	3
4.4 Trie 字典樹	3
4.5 AC 自動機	4
5 圖論: 最大流	4
5.1 BFS 與拓樸排序	4
5.2 DFS	5
5.3 Dijkstra 單源最短路徑	5
5.4 SPFA 單源最短路徑	5
5.5 Floyd-Warshall 多源最短路徑	6
5.6 二分圖判定 (著色問題)	6
5.7 割點與雙連通分量	6
5.8 匈牙利演算法	7
5.9 KM 演算法	7
6 幾何	7
6.1 點與向量基礎 (Point & Vector)	7
6.2 判斷線段、矩形相交 (Segment Intersection)	8
6.3 點與線段關係 (Point & Line)	8
6.4 任意多邊形面積	8
6.5 凸包 Convex Hull	8
7 樹論	9
7.1 Disjoint Set	9
7.2 Kruskal 最小生成樹	9
8 DP: 分錢幣問題, Kadane's algo 找最大子序列問題	9
8.1 01 Knapsack 背包	9
8.2 LCS 最長共同子序列 (Sequence)	9
8.3 區間 DP 概念 (最長迴文子序列)	10
8.4 跳格子	10
8.5 LIS / LDS 最長遞增/遞減子序列	11
9 Greedy	11
9.1 一維區間全覆蓋	11
9.2 間格跳躍	11
9.3 最大不重疊區間	11
9.4 最小覆蓋區間	12
9.5 區間選點問題	12
9.6 最小化最大延遲問題	12
9.7 Huffman 樹	12
9.8 任務調度問題	12
9.9 多機調度問題	12
10 MISC	13
10.1 模板	13
10.2 一些概念	13
10.3 C++ 函式庫	13

1 你是否有...

1.1 嘗試...

- 再讀一遍題目
- 懷疑你認為「理所當然」的直覺?
- 試著寫下直覺的規律? (有助於觀察)
- 換個方向思考?
- 暴力解?
- 貪心解?
- 對答案二分搜?
- 動態規劃 (DP)?
- 狀態建模, 把問題轉化成圖論?
- 只在奇數 index 反轉狀態?
- 用乘法取代除法? ($/2 = *2^{-1}$, 模逆元)

1.2 檢查...

- 確定沒漏看敘述?
- 確認數值範圍? (溢位, 精度位數, 陣列大小...)
- 不同的陣列/字串使用同個 index 導致超出範圍?
- 模板抄錯?
- 多筆資料之間初始化?
- 題目額外的敘述, 暗示?
- 未定義行為? (陣列越界, 未初始化)
- overflow? (int -> long long)
- 非 void function 忘記 return?
- 浮點數誤差? (極小的 eps, 是否需要 long double)
- 隱轉換? (int*int 爆了後才存到 long long)
- signed 與 unsigned int 之間的 compare?

2 資料結構

2.1 BIT (Fenwick Tree)

用法: 支援單點修改與區間求和。必須是 1-based index。
時間複雜度: $\mathcal{O}(\log n)$

```

1 struct BIT {
2     int n;
3     vector<long long> tree; // 使用 long long 避免區間總和溢位
4
5     void init(int sz){
6         n = sz;
7         tree.assign(sz+1, 0);
8     }
9
10    // lowbit: 取得二進位中最右邊的 1
11    inline int lowbit(int x) {
12        return x & (-x);
13    }
14
15    // 單點修改: 在位置 x 加上了 val
16    void add(int x, long long val) {
17        for (; x <= n; x += lowbit(x)) {
18            tree[x] += val;
19        }
20    }
21
22    // 前綴查詢: 計算 [1, x] 的總和
23    long long query(int x) {
24        long long sum = 0;
25        for (; x > 0; x -= lowbit(x)) {
26            sum += tree[x];
27        }
28        return sum;
29    }
30
31    // 區間查詢: 計算 [l, r] 的總和 (前綴和相減概念)
32    long long query(int l, int r) {
33        return query(r) - query(l - 1);
34    }
35 };

```

2.2 Segment Tree 線段樹

用法：給定 n 筆資料查詢與 m 次範圍更新，解決多次範圍更新的問題，找區間和或最大/小值。必須是 1-based index。

時間複雜度：建樹 $\mathcal{O}(N)$ 查詢/更新 $\mathcal{O}(\log N)$

```
1 using ll = long long;
2
3 vector<ll> arr, sum, add; //宣告全域，arr 是原始數據，sum 是範圍累加和，add
  是 lazy tag 儲存的值
4
5 void up(int i){ // 往上加回去
6     sum[i] = sum[i*2] + sum[i*2+1];
7 }
8
9 void lazy(int i, ll v, int n){ // 懶標記 (lazy tag)，暫存當前覆蓋範圍
10    sum[i] += v*n;
11    add[i] += v;
12 }
13
14 void down(int i, int ln, int rn){ // 往下分發 lazy tag
15     if(add[i] != 0){
16         lazy(i*2, add[i], ln);
17         lazy(i*2+1, add[i], rn);
18         add[i] = 0;
19     }
20 }
21
22 void build(int l, int r, int i){ // 遞迴式初始化 (init)
23     if(l == r) sum[i] = arr[l]; // 如果區間只剩一個數值，就直接賦值
24     else{
25         int mid = l + (r-l)/2; // 找中點
26         build(l, mid, i*2); // 左半邊 build
27         build(mid+1, r, i*2+1); // 右半邊 build
28         up(i); // 自己 = 左 + 右 區間和
29     }
30     add[i] = 0; // 初始化所有 add[i] = 0
31 }
32
33 void update(int jobl, int jobr, ll jobv, int l, int r, int i){ // 更新區間數
  值 (jobl ~ jobr 加上 jobv)，用 l, r, i 判斷範圍
34     if(jobl <= l && r <= jobr) lazy(i, jobv, r-l+1); // 如果範圍被包的話就直接
  lazy
35     else{
36         int mid = l + (r-l)/2;
37         down(i, mid-l+1, r-mid); // 記得往下分發 lazy tag
38         if(jobl <= mid) update(jobl, jobr, jobv, l, mid, i*2); // 判斷左右區
  段是否需要更新
39         if(jobr > mid) update(jobl, jobr, jobv, mid+1, r, i*2+1);
40         up(i); // 最後往上加總回去更新 sum
41     }
42 }
43
44 ll query(int jobl, int jobr, int l, int r, int i){
45     if(jobl <= l && r <= jobr) return sum[i]; // 如果包範圍就直接 return 回去
46
47     int mid = l + (r-l)/2;
48     down(i, mid-l+1, r-mid); // 記得往下分發 lazy tag
49     ll total = 0;
50     if(jobl <= mid) total += query(jobl, jobr, l, mid, i*2);
51     if(jobr > mid) total += query(jobl, jobr, mid+1, r, i*2+1);
52     return total; // 因為只是 query，沒有修改，所以不用 up 更新回去
53 }
54
55 int main() {
56     int n, m;
57     cin >> n >> m;
58     arr.assign(n+1, 0);
59     sum.assign(4*(n+1), 0); // 大小必須開到 4 倍 n+1
60     add.assign(4*(n+1), 0);
61
62     for(int i=1; i<=n; ++i) cin >> arr[i];
63     build(1, n, 1);
64     int op;
65     ll x, y, k;
66     while(m--){
67         cin >> op;
68         if(op == 1){
69             cin >> x >> y >> k;
70             update(x, y, k, 1, n, 1);
71         }
72         else{
73             cin >> x >> y;
74             cout << query(x, y, 1, n, 1) << '\n';
75         }
76     }
77     return 0;
78 }
```

3 數論：尤拉公式，數論分塊，Sprague-Grundy 函數，排容原理

3.1 快速冪

用法：計算 $a^b \pmod m$ 。

時間複雜度： $\mathcal{O}(\log b)$

```
1 // 計算 (a^b) % mod
2 // 競賽中 mod 通常為 1e9+7 或 998244353
3 long long fast_pow(long long a, long long b, long long mod) {
4     long long res = 1;
5     a %= mod; // 預防初始底數 a 大於等於 mod
6
7     while (b > 0) {
8         // 如果當前次方數是奇數，把當前的 a 乘進答案裡
9         if (b & 1) res = res * a % mod;
10        a = a * a % mod; // 底數平方
11        b >>= 1; // 次方數除以 2
12    }
```

```
12    }
13    return res;
14 }
```

3.2 模逆元與費馬小定理

用法：定義：若 p 為質數，且整數 a 與 p 互質，則 $a^{p-1} \equiv 1 \pmod p$ 。邏輯應用：兩邊同除以 a 可得 $a^{p-2} \equiv a^{-1} \pmod p$ 。

這代表計算 $(x/a) \pmod p$ 等同於計算 $(x \cdot a^{p-2}) \pmod p$ 。

時間複雜度： $\mathcal{O}(\log p)$

```
1 // 模逆元 (Modular Multiplicative Inverse)
2 // 條件：mod 必須是質數 (利用費馬小定理)
3 // 應用：計算 (x / y) % mod 等同於計算 (x * inv(y, mod)) % mod
4 long long inv(long long a, long long mod) {
5     return fast_pow(a, mod - 2, mod);
6 }
7
8 // 應用範例：計算 (a / b) % mod
9 long long mod_div(long long a, long long b, long long mod) {
10    return (a % mod) * mod_inv(b, mod) % mod;
11 }
```

3.3 質數檢查

用法：單次詢問大數用 isPrime；若有大量詢問則建質數表。

時間複雜度： $\mathcal{O}(\sqrt{N})$

```
1 // 1. 單次質數檢驗
2 // 適用時機：只詢問極少數幾個數字，且數字可能很大 (<= 1e14)
3 bool isPrime(long long n) {
4     if (n < 2) return false;
5     if (n == 2 || n == 3) return true;
6     if (n % 2 == 0 || n % 3 == 0) return false;
7
8     // 效能優化：除了 2 和 3，質數一定出現在 6k-1 或 6k+1
9     for (long long i = 5; i * i <= n; i += 6) {
10        if (n % i == 0 || n % (i + 2) == 0) return false;
11    }
12    return true;
13 }
14
15 // =====
16
17 // 2. 埃拉托斯特尼篩法 (Sieve of Eratosthenes)
18 // 適用時機：需要大量查詢 1 - 10^7 內的質數
19 const int MAXN = 1e7 + 10;
20 vector<int> primes;
21 bool is_prime[MAXN];
22
23 void sieve(int n) {
24     fill(is_prime, is_prime + n + 1, true);
25     is_prime[0] = is_prime[1] = false;
26
27     for (int i = 2; i <= n; ++i) {
28         if (is_prime[i]) {
29             primes.push_back(i); // 收集質數
30
31             // 重要優化：從 i * i 開始篩，因為 i * 2, i * 3... 在之前已經被 2,
32             3... 篩過了
33             // 注意型態轉型避免 i * i 爆 int
34             for (long long j = (long long)i * i; j <= n; j += i) {
35                 is_prime[j] = false;
36             }
37         }
38     }
```

3.4 大數乘法

用法：未知長度的大數相乘

時間複雜度： $\mathcal{O}(N \times M)$

```
1 // 1. 大數 x 小整數 (BigInt * int)
2 // 適用場景：算階乘、連續乘上範圍在 int 內的數值
3 // 時間複雜度： $\mathcal{O}(N)$ 
4 vector<int> multiply_small(const vector<int>& a, int b) {
5     if (b == 0 || (a.size() == 1 && a[0] == 0)) return {0};
6
7     vector<int> c;
8     long long carry = 0; // 使用 long long 避免溢位
9
10    for (size_t i = 0; i < a.size() || carry > 0; i++) {
11        if (i < a.size()) carry += 1LL * a[i] * b;
12        c.push_back(carry % 10);
13        carry /= 10;
14    }
15    return c;
16 }
17
18 // 2. 大數 x 大數 (BigInt * BigInt)
19 // 適用場景：兩個長度未知的超大整數相乘
20 // 時間複雜度： $\mathcal{O}(N * M)$ 
21 vector<int> multiply_big(const vector<int>& a, const vector<int>& b) {
22     if (a.empty() || b.empty()) return {0};
23     if ((a.size() == 1 && a[0] == 0) || (b.size() == 1 && b[0] == 0)) return {0};
24
25     // 結果的最大可能長度為兩者長度相加
26     vector<int> c(a.size() + b.size(), 0);
27
28     for (size_t i = 0; i < a.size(); i++) {
29         long long carry = 0;
30         for (size_t j = 0; j < b.size() || carry > 0; j++) {
31             // 注意：要加上原本在 c[i+j] 已經算出的值
32             long long cur = c[i + j] + 1LL * a[i] * b[j] : 0)
33             + carry;
34             c[i + j] = cur % 10;
35             carry = cur / 10;
36         }
37     }
```

```

34         carry = cur / 10;
35     }
36 }
37
38 // 移除多餘的前導零 (例如 00 變成 0)
39 while (c.size() > 1 && c.back() == 0) {
40     c.pop_back();
41 }
42
43 return c;
44 }
45
46 // 輔助函式：將字串轉換為大數陣列 (反向存入)
47 vector<int> string_to_bigint(const string& s) {
48     vector<int> res;
49     for (int i = s.length() - 1; i >= 0; i--) {
50         res.push_back(s[i] - '0');
51     }
52     return res;
53 }
54
55 // 輔助函式：印出大數 (反向印出)
56 void print_bigint(const vector<int>& a) {
57     for (int i = a.size() - 1; i >= 0; i--) {
58         cout << a[i];
59     }
60     cout << "\n";
61 }

```

4 字串

4.1 KMP 字串匹配

用法：尋找 `pattern` 出現位置。`pi` 陣列可求最小循環節。必須是 0-based index。

時間複雜度： $\mathcal{O}(N + M)$

```

1 // pi 陣列 (Longest Prefix Suffix): pi[i] 代表 pattern[0..i] 的最長相同前後綴
  長度
2 vector<int> build_pi(const string& s) {
3     int m = s.length();
4     vector<int> pi(m, 0);
5     for (int i = 1, j = 0; i < m; ++i) {
6         // 如果不匹配, 就利用 pi 陣列往回跳
7         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
8         if (s[i] == s[j]) j++;
9         pi[i] = j;
10    }
11    return pi;
12 }
13
14 // 可用 pi 陣列尋找一個字串的最小循環節
15 int getMinimumPeriod(const string& s) {
16     int n = s.length();
17     if (n == 0) return 0;
18     vector<int> pi = build_pi(s);
19
20     // 取得整個字串的最長相同前後綴長度
21     int maxPrefixLength = pi[n - 1];
22
23     // 計算可能的最小循環節長度
24     int period = n - maxPrefixLength;
25
26     // 檢查總長度是否能被 period 整除
27     if (maxPrefixLength > 0 && n % period == 0) {
28         return period;
29     }
30
31     // 若無法整除, 代表沒有更小的週期, 最小循環節為字串本身
32     return n;
33 }
34 // 最終, 最小循環節長度即為回傳的 period -> s.substr(0, period);
35
36 // 執行字串匹配
37 vector<int> kmp(const string& text, const string& pattern) {
38     vector<int> res;
39     if (pattern.empty()) return res;
40
41     vector<int> pi = build_pi(pattern);
42     for (int i = 0, j = 0; i < text.length(); ++i) {
43         while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
44         if (text[i] == pattern[j]) j++;
45
46         if (j == pattern.length()) { // 完全匹配
47             res.push_back(i - j + 1); // 紀錄起始 index
48             j = pi[j - 1]; // 繼續尋找下一個可能的匹配
49         }
50     }
51     return res; // 回傳的這個 vector 包含所有完全匹配的 index 位置
52 }

```

4.2 Z-Algorithm 字串匹配

用法：`z[i]` 代表 S 與 $S[i \dots N - 1]$ 的最長共同前綴。必須是 0-based index。

時間複雜度： $\mathcal{O}(N)$

```

1 // z[i] 代表 s[0..n-1] 與 s[i..n-1] 的最長共同前綴 (LCP) 長度
2 // 若要找 pattern 在 text 中的位置, 可傳入 s = pattern + "$" + text
3 vector<int> z_function(const string& s) {
4     int n = s.length();
5     vector<int> z(n, 0);
6     // l, r 記錄目前「最靠右」的匹配區間 [l, r]
7     for (int i = 1, l = 0, r = 0; i < n; ++i) {
8         // 如果 i 在區間內, 可以直接利用對稱性 (i - l) 抄答案
9         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
10
11         // 暴力往後擴展 (因為有前面的加速, 這裡實際上只會做很少次)

```

```

12         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
13             z[i]++;
14         }
15
16         // 如果新匹配的區間超過了原有的 r, 更新 [l, r] 視窗
17         if (i + z[i] - 1 > r) {
18             l = i;
19             r = i + z[i] - 1;
20         }
21     }
22     return z;
23 }
24
25 string pattern = "aba";
26 string text = "abacaba";
27
28 string s = pattern + "$" + text;
29
30 vector<int> z = z_function(s);
31
32 for(int i = 0; i < s.size(); i++) {
33     if(z[i] == pattern.size()) {
34         cout << "found at " << i - pattern.size() - 1 << "\n";
35     }
36     /*
37     found at 0
38     found at 4
39     */
40 }

```

4.3 Manacher 迴文判斷

用法：求最長迴文子字串。預處理字串塞入 # 將所有迴文轉為奇數長度。

時間複雜度： $\mathcal{O}(N)$

```

1 // 求最長迴文
2 struct Manacher {
3     vector<int> p; // p[i] 記錄以 i 為中心的最長迴文半徑
4     string t; // 轉換後的字串
5
6     Manacher(const string& s) {
7         // 預處理：將 "abc" 變成 "#a#b#c#" 來統一奇偶長度
8         // 頭尾放不同的字元 ($) 和 ~ 可省去邊界判斷
9         t = "$#";
10        for (char c : s) { t += c; t += '#'; }
11        t += '~';
12
13        int n = t.length();
14        p.assign(n, 0);
15        int c = 0, r = 0; // c: 目前最靠右的迴文中心, r: 該迴文的右界
16
17        for (int i = 1; i < n - 1; i++) {
18            int mirror = 2 * c - i; // i 關於 c 的對稱點
19
20            // 如果 i 在迴文右界 r 內, 可以直接抄對稱點的答案
21            if (r > i) p[i] = min(r - i, p[mirror]);
22
23            // 暴力往外擴展半徑
24            while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) {
25                p[i]++;
26            }
27
28            // 如果這個新迴文的右界超過了 r, 更新 c 和 r
29            if (i + p[i] > r) {
30                c = i;
31                r = i + p[i];
32            }
33        }
34
35        // 回傳原字串中的最長迴文長度
36        int get_max_len() {
37            int max_len = 0;
38            for (int len : p) max_len = max(max_len, len);
39            return max_len;
40        }
41
42        // 回傳所有出現的迴文, 包含位置不同的重複字串
43        // 如果要不重複就把 vector 改成 set 進行去重, 再 for 輸出就好
44        vector<string> get_all_palindromes(const string& s) {
45            vector<string> result;
46            // 遍歷所有可能的迴文中心
47            for (int i = 1; i < t.length() - 1; i++) {
48                // p[i] 是以 i 為中心的最大迴文長度
49                // 每次往內縮 2 (砍掉頭尾字元), 即可得到以此為中心的所有較短迴文
50                for (int len = p[i]; len > 0; len -= 2) {
51                    // 神奇的公式: (i - len) / 2 可以精準對應到原字串 s 的起點索引
52                    int start_idx = (i - len) / 2;
53                    result.push_back(s.substr(start_idx, len));
54                }
55            }
56            return result;
57        }
58    }
59
60    // 僅計算所有迴文子字串的「總數量」
61    long long count_all_palindromes() {
62        long long total = 0;
63        for (int i = 1; i < t.length() - 1; i++) {
64            // 對於中心 i, 包含的迴文數量恰好是 ceil(p[i] / 2)
65            total += (p[i] + 1) / 2;
66        }
67        return total;
68    }
69 };

```

4.4 Trie 字典樹

用法：字串前綴統計或最大 XOR 問題。必須是 0-based index。

時間複雜度： $\mathcal{O}(|S|)$

```

1 struct Trie {
2     // 如果是字元 size=26, 如果是二進制 XOR size=2
3     int tree[MAXN][size] = {};
4     int passed[MAXN] = {};
5     int stop[MAXN] = {};
6     // 以上陣列的都跳過 index 0
7     int cnt = 1;
8
9     void init(){
10         for(int i=1; i<cnt; ++i){
11             for(int j=0; j<size; ++j) tree[i][j] = 0;
12             passed[i] = 0;
13             stop[i] = 0;
14         }
15         cnt = 1;
16     }
17
18     void insert(string word){
19         int curr = 1;
20         ++passed[curr]; // 經過點, ++passed
21         for(int i=0; path; i<word.length(); ++i){
22             path = word.at(i) - 'a'; // 求出下一個位置的 index
23             if(tree[curr][path] == 0) tree[curr][path] = ++cnt; // 如果當前位置走
24             到目標位置不存在, 就用 ++cnt 的值
25             curr = tree[curr][path]; // 繼續往下走
26             ++passed[curr]; // 記得經過就要 ++passed
27         }
28         ++stop[curr]; // 最後停住的地方要 ++stop
29     }
30
31     int search(string word){
32         int curr = 1;
33         for(int i=0; path; i<word.length(); ++i){
34             path = word.at(i) - 'a';
35             if(tree[curr][path] == 0) return 0; // 如果從 curr 往 path 的路徑不存
36             在, 代表搜尋的 word 不存在
37             curr = tree[curr][path]; // 否則繼續往下走
38         }
39         return stop[curr]; // 最後回傳在 curr 位置停下 (stop) 的次數
40     }
41
42     int prefixNumber(string word){
43         int curr = 1;
44         for(int i=0; path; i<word.length(); ++i){
45             path = word.at(i) - 'a';
46             if(tree[curr][path] == 0) return 0;
47             curr = tree[curr][path];
48         }
49         return passed[curr]; // 和 search 方法一模一樣, 只是回傳的對象變成 passed
50     }
51
52     void erase(string word){
53         if(search(word) == 0) return; // 如果找不到字就不管他
54         int curr = 1;
55         --passed[curr];
56         for(int i=0; path; i<word.length(); ++i){
57             path = word.at(i) - 'a';
58             if(--passed[tree[curr][path]] == 0){ // 如果 curr 往 path 的路徑上, p
59             值為 1, 代表後面的東西都不需要了
60             tree[curr][path] = 0; // 將 curr 不再連通 path, 直接捨棄後面所有的東
61             西 (因為都會被減成 p=0)
62             return;
63         }
64         curr = tree[curr][path]; // 否則繼續往下走, 注意後面不需要
65         再--passed[curr], 因為在 if 區塊我們已經操作完了
66     }
67
68     --stop[curr]; // 最後--stop[curr]
69 }
70
71 };

```

4.5 AC 自動機

用法: 多字串匹配 (Trie + KMP), 必須是 0-based index。

時間複雜度: $\mathcal{O}(N + M)$

```

1 struct AC_Automaton {
2     static const int MAXN = 1e5 + 10;
3     int tree[MAXN][size]; // size 是字元 26, 二進制 2
4     int fail[MAXN];
5     int stop[MAXN]; // 記錄有幾個字串在此結束
6     int cnt;
7
8     void init() {
9         cnt = 0;
10        new_node(); // root is 0
11    }
12
13    int new_node() {
14        fill(tree[cnt], tree[cnt] + 26, 0);
15        fail[cnt] = stop[cnt] = 0;
16        return cnt++;
17    }
18
19    // 1. 將所有關鍵字插入 Trie
20    void insert(const string& s) {
21        int curr = 0;
22        for (char c : s) {
23            int path = c - 'a';
24            if (!tree[curr][path]) {
25                tree[curr][path] = new_node();
26            }
27            curr = tree[curr][path];
28        }
29        stop[curr]++;
30    }
31
32    // 2. 建立 Fail 指標與字典圖優化 (使用 BFS)
33    void build() {
34        queue<int> q;
35        // 將 root (0) 的所有實際存在的第一層子節點推入 queue

```

```

36        for (int i = 0; i < 26; ++i) {
37            if (tree[0][i]) {
38                fail[tree[0][i]] = 0;
39                q.push(tree[0][i]);
40            }
41        }
42
43        while (!q.empty()) {
44            int u = q.front();
45            q.pop();
46
47            for (int i = 0; i < 26; ++i) {
48                if (tree[u][i]) {
49                    // 若子節點存在, 它的 fail 指向父節點 fail 的對應子節點
50                    fail[tree[u][i]] = tree[fail[u]][i];
51                    q.push(tree[u][i]);
52                } else {
53                    // 優化: 若子節點不存在, 直接把這條路連向 fail 的對應節點
54                    // 這樣搜尋時就絕對不會遇到死胡同, 也不用往回跳
55                    tree[u][i] = tree[fail[u]][i];
56                }
57            }
58        }
59    }
60
61    // 3. 搜尋文章, 回傳所有關鍵字出現的總次數
62    int query(const string& text) {
63        int curr = 0;
64        int res = 0;
65        for (char c : text) {
66            curr = tree[curr][c - 'a']; // 感謝字典圖優化, 無腦往下走即可
67
68            // 順着 fail 指標往上收集沿途所有匹配成功的字串 (例如配對到 "she",
69            // 同時也配對到 "he")
70            int temp = curr;
71            while (temp != 0 && stop[temp] != -1) {
72                res += stop[temp];
73                stop[temp] = -1; // -1 是優化: 已經算過的關鍵字不要重複算
74                temp = fail[temp];
75            }
76            return res;
77        }
78    };

```

5 圖論: 最大流

需注意:

是否為全連通圖 (connected, 只有一個 component), 對於連通圖要找 region 可以用 Euler's theorem $V-E+R=2$

undirected 無自環

directed

weighted 有權重 (點或邊)

multigraph 兩點之間有多條直接的路徑

有向圖的 outdegree, indegree, 無向圖的 degree

表示圖的方式: Adjacency matrix, adjacency list

5.1 BFS 與拓樸排序

用法: 無權圖最短路徑 / 拓樸排序計算 In-degree 尋找相關順序, 可判斷是否有環。

時間複雜度: $\mathcal{O}(V + E)$

```

1 struct BFS_Topo {
2     int n;
3     vector<vector<int>> adj; // adjacency list, 避免爆空間
4     vector<int> in_degree; // 紀錄入度, 用於拓樸排序
5
6     void init(int _n) {
7         n = _n;
8         adj.assign(n + 1, vector<int>());
9         in_degree.assign(n + 1, 0);
10    }
11
12    // 有向圖, u 指向 v (u 必須在 v 之前完成)
13    void add_edge(int u, int v) {
14        adj[u].push_back(v);
15        in_degree[v]++; // v 被 u 指到, 入度 +1
16    }
17
18    // 1. 拓樸排序 (Kahn's Algorithm)
19    // 回傳拓樸排序的結果。若回傳的 vector 大小小於 n, 代表圖中有環
20    vector<int> topo_sort() {
21        queue<int> q;
22        vector<int> res;
23
24        // 步驟一: 將所有入度為 0 (沒有前置條件) 的點加入 queue
25        for (int i = 1; i <= n; ++i) {
26            if (in_degree[i] == 0) q.push(i);
27        }
28
29        // 步驟二: 開始 BFS
30        while (!q.empty()) {
31            int u = q.front();
32            q.pop();
33            res.push_back(u);
34
35            // 拔掉 u 連出去的所有邊
36            for (int v : adj[u]) {
37                in_degree[v]--; // v 的前置條件少了一個
38                if (in_degree[v] == 0) { // 如果前置條件都滿足了, 就推入 queue
39                    q.push(v);
40                }

```

```

41     }
42 }
43 return res; // 若 res.size() < n, 說明有環, 無法完成拓樸排序
44 }
45
46 // 2. 基礎 BFS: 無權圖的單源最短路徑
47 vector<int> get_dist(int start) {
48     vector<int> dist(n + 1, -1);
49     queue<int> q;
50
51     dist[start] = 0;
52     q.push(start);
53
54     while (!q.empty()) {
55         int u = q.front();
56         q.pop();
57         for (int v : adj[u]) {
58             if (dist[v] == -1) { // 沒走過
59                 dist[v] = dist[u] + 1;
60                 q.push(v);
61             }
62         }
63     }
64     return dist;
65 }
66 };

```

5.2 DFS

用法: 用遞迴走訪。

時間複雜度: $\mathcal{O}(V + E)$

```

1 struct DFS {
2     int n;
3     vector<vector<int>>> adj;
4     vector<bool> vis;
5
6     void init(int _n) {
7         n = _n;
8         adj.assign(n + 1, vector<int>());
9         vis.assign(n + 1, false);
10        subtree_size.assign(n + 1, 0);
11    }
12
13    void add_edge(int u, int v) {
14        adj[u].push_back(v);
15        adj[v].push_back(u); // 無向圖就兩個都加
16    }
17
18    // 基礎 DFS: 走訪與找連通塊
19    void dfs(int u) {
20        vis[u] = true;
21        // 如有需要, 處理抵達 u 時的邏輯 (Pre/In/Post-order)
22
23        for (int v : adj[u]) {
24            if (!vis[v]) {
25                dfs(v);
26            }
27        }
28    }
29 };
30
31 /*
32 補充 1:
33 當題目節點數量 N <= 10, 需要找一條 "最短路徑", 且每個邊的最大 degree 只有 2
34 可以使用 DFS 進行枚舉, 從各個節點開始跑, 對所有可能的路徑跑 DFS
35 邊跑邊更新最長路徑答案, 跑完是 O(N!) -> N=10 -> 小於 400 萬, 能跑
36 */
37
38 /*
39 補充 2:
40 如果發現題目要找樹的直徑時, 可以使用兩次 dfs 求解 (前提是樹邊權不能是負的)
41 如: 樹上最遠兩點距離? 從一個人開始通知, 每秒傳到相鄰節點, 最長多久可以全收到? 路
    徑唯一最長鏈?
42 */
43 vector<int> adj[MAXN];
44 int max_dist;
45 int farthest_node;
46
47 void dfs(int cur, int parent, int dist) {
48     if (dist > max_dist) {
49         max_dist = dist;
50         farthest_node = cur;
51     }
52     for (int &next : adj[cur]) {
53         if (next != parent) dfs(cur->next, cur, dist+1);
54     }
55 }
56
57 void solve() {
58     int n, e;
59     cin >> n >> e;
60     for (int i=0; i<e; ++i) { // 讀入圖
61         int a, b;
62         cin >> a >> b;
63         adj[a].push_back(b);
64         adj[b].push_back(a);
65     }
66     max_dist = -1;
67     dist(1, -1, 0); // 一開始從任意點第一次 DFS
68
69     int first_farthest = farthest_node;
70     max_dist = -1;
71     dist(first_farthest, -1, 0);
72     // 最長距離為 max_dist, 點: first_farthest, farthest_node
73 }

```

5.3 Dijkstra 單源最短路徑

用法: 找單個起點的最短路徑, 不支援負權邊, 支援自環。

時間複雜度: $\mathcal{O}((V + E) \log V)$

```

1 struct Dijkstra {
2     // 定義 pii 為 pair< 距離, 節點 >, 預設由小到大排序 (pq 用)
3     using pii = pair<long long, int>;
4     const int MAXN = 2e5 + 10;
5     const long long INF = 1e18; // 使用 1e18 避免相加時 long long 溢位
6
7     vector<pair<int, long long>> adj[MAXN]; // adj[u] = {v, weight}
8     long long dist[MAXN]; // dist[i] 為起點到 i 的距離
9     int n;
10
11     // 初始化: 傳入節點數, 清空圖
12     void init(int _n) {
13         n = _n;
14         for (int i = 0; i <= n; ++i) {
15             adj[i].clear();
16         }
17     }
18
19     void add_edge(int u, int v, long long w) {
20         adj[u].push_back({v, w});
21         // 若為無向圖, 須加上 adj[v].push_back({u, w});
22     }
23
24     void solve(int start) {
25         fill(dist, dist + n + 1, INF); // 將距離陣列初始化為無限大
26         // pq 存起點到某一點的距離, 用距離的 Min-Heap
27         priority_queue<pii, vector<pii>, greater<pii>> pq;
28
29         dist[start] = 0;
30         pq.push({0, start});
31
32         while (!pq.empty()) {
33             long long d = pq.top().first;
34             int u = pq.top().second;
35             pq.pop();
36
37             // 懶惰刪除 (Lazy Deletion): 如果取出時發現已經有更好的距離, 直接丟棄
38             if (d > dist[u]) continue;
39
40             for (auto& edge : adj[u]) {
41                 int v = edge.first;
42                 long long w = edge.second;
43
44                 // 鬆弛 (Relax)
45                 if (dist[u] + w < dist[v]) {
46                     dist[v] = dist[u] + w;
47                     pq.push({dist[v], v});
48                 }
49             }
50         }
51     }
52 };

```

5.4 SPFA 單源最短路徑

用法: 也是找單個起點的最短路徑, 但支援負權邊。只在“可能有負邊”或“必須檢查負環”時才用

時間複雜度: 預期 $\mathcal{O}(V + E)$, 最差 $\mathcal{O}(VE)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1005;
5 const long long INF = 1e18;
6
7 struct Edge {
8     int v;
9     long long w;
10 };
11
12 vector<Edge> G[MAXN]; // adjacency list 存圖
13 long long dist[MAXN]; // 起點到任意點的距離
14 int n, m; // n 個點, m 個邊
15
16 bool SPFA(int start) { // 回傳 false 代表圖中存在負環; 回傳 true 代表最短路徑計算成功
17     bool inq[MAXN]; // inq[i] 記錄點 i 目前是否在 queue 裡面
18     int cnt[MAXN]; // cnt[i] 記錄點 i 進入 queue 的總次數
19     queue<int> q;
20
21     fill(dist, dist + n + 1, INF);
22     memset(inq, false, sizeof(inq));
23     memset(cnt, 0, sizeof(cnt));
24
25     dist[start] = 0; // 從 start 節點開始
26     q.push(start);
27     inq[start] = true;
28     cnt[start] = 1;
29
30     while (!q.empty()) {
31         int u = q.front();
32         q.pop();
33         inq[u] = false; // 拿出來後就不在 queue 裡了
34
35         for (auto& edge : G[u]) {
36             int v = edge.v;
37             long long w = edge.w;
38
39             // 鬆弛操作 (relax)
40             if (dist[u] + w < dist[v]) {
41                 dist[v] = dist[u] + w;
42
43                 // 如果被鬆弛的點不在 queue 裡面, 就把它加進去
44                 if (!inq[v]) {
45                     q.push(v);
46                     inq[v] = true;
47                     cnt[v]++;
48

```



```

49         if (cnt[v] >= n) return false; // 【關鍵】一個點入隊次數
           >= n (點總數)，必定有負環
50     }
51 }
52 }
53 }
54 return true;
55 }

```

5.5 Floyd-Warshall 多源最短路徑

用法：求任意兩點之間距離。利用滾動陣列降維，**k** 必須在最外層迴圈。注意避開 **INF** 加上負權邊的陷阱。

時間複雜度：時間 $\mathcal{O}(V^3)$ ，空間 $\mathcal{O}(V^2)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 505;
5 const long long INF = 1e18;
6 long long dist[MAXN][MAXN];
7 int n, m;
8
9 void init() {
10     for (int i = 1; i <= n; ++i) {
11         for (int j = 1; j <= n; ++j) {
12             dist[i][j] = (i == j) ? 0 : INF;
13         }
14     }
15 }
16
17 // 注意：讀邊時若為多重邊 (Multigraph, 即 A->B 不只一條 direct path)，需寫成
18 // dist[u][v] = min(dist[u][v], w);
19
20 void floyd() {
21     // 中繼點 k 必須在最外層！
22     for (int k = 1; k <= n; ++k) {
23         for (int i = 1; i <= n; ++i) {
24             for (int j = 1; j <= n; ++j) {
25                 // 必須確認 i->k 和 k->j 都是通的
26                 if (dist[i][k] < INF && dist[k][j] < INF) {
27                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
28                 }
29             }
30         }
31     }
32 }

```

5.6 二分圖判定 (著色問題)

用法：規定相鄰節點必須塗上不同顏色，如果在著色過程中，發現相鄰節點已經被塗上相同的顏色，則代表此圖包含奇數環 Odd Cycle，不是二分圖。已考量到不連通圖的情況

時間複雜度： $\mathcal{O}(V + E)$

```

1 struct BipartiteCheck {
2     int n;
3     vector<vector<int>>> adj;
4     vector<int> color; // 0: 未著色, 1: 顏色 A, -1: 顏色 B
5
6     void init(int _n) {
7         n = _n;
8         adj.assign(n + 1, vector<int>());
9         color.assign(n + 1, 0); // 初始化全為未著色
10    }
11
12    void add_edge(int u, int v) {
13        adj[u].push_back(v);
14        adj[v].push_back(u); // 二分圖判斷通常針對無向圖
15    }
16
17    bool solve() {
18        bool is_bipartite = true;
19
20        // 圖可能有多个連通分量 (Components)，必須確保每個節點都被檢查到
21        for (int i = 1; i <= n; ++i) {
22            if (color[i] == 0) { // 遇到未著色的節點，當作新 Component 的起點
23                queue<int> q;
24                q.push(i);
25                color[i] = 1; // 塗上第一種顏色
26
27                while (!q.empty()) {
28                    int u = q.front();
29                    q.pop();
30
31                    for (int v : adj[u]) {
32                        if (color[v] == 0) {
33                            // 相鄰節點未著色，塗上相反的顏色 (-color[u]) 並加入 queue
34                            color[v] = -color[u];
35                            q.push(v);
36                        } else if (color[v] == color[u]) {
37                            // 相鄰節點已著色，且與自己同色 -> 產生奇數環，非二分圖
38                            is_bipartite = false;
39                            return false; // 提早結束，不需繼續檢查
40                        }
41                    }
42                }
43            }
44        }
45        return is_bipartite;
46    }
47 };

```

5.7 割點與雙連通分量

用法：處理網路穩定性、單點故障分析。透過 DFS 一次遍歷找出所有的割點以及 **v-BCC**。由於一個割點可能同時屬於多個 BCC，我們利用 **stack** 記錄 DFS 走過的點，當發現割點條件成立時，再把點從 **stack** 彈出形成 BCC。

時間複雜度： $\mathcal{O}(V + E)$

```

1 /*
2 dfn[u]: DFS 時，節點 u 被拜訪到的「時間戳記」或「順序」。一旦給定就不會再改變。
3
4 low[u]: 從節點 u 的子樹出發，在不經過 u 的那條父節點邊 的前提下，透過
5 back-edge 能到達的所有節點中，最小的 dfn 值。
6
7 /*
8 割點 (Articulation Point):
9 定義：在無向連通圖中，如果移除某個節點及其所有相連的邊後，圖被切斷成兩個或多個不
10 相連的部分，該點就是割點。
11 用途：尋找網路、電路或交通系統中的「單點故障」瓶頸。只要這個點壞了，整個系統就會癱
12 瘓。
13 判斷條件：
14 若 u 是 DFS 的根節點 (Root)，且它有 2 個以上的子樹，則 u 是割點。
15 若 u 不是根節點，且存在至少一個子節點 v 滿足 low[v] >= dfn[u]。這代表 v 及其子孫
16 「最多只能爬回 u 自己」，無法繞過 u 抵達更上層的祖先。如果把 u 拔掉，v 就會跟上面斷
17 聯，因此 u 是割點。
18
19 點雙連通分量 (Vertex Biconnected Component):
20 定義：一個不包含任何割點的「極大連通子圖」。在 v-BCC 中，任意兩點之間至少有兩條點
21 不重複的路徑。
22 用途：用來縮點。把一個個 v-BCC 視為一個大節點，可以將複雜的圖形轉化為「樹」的結
23 構，進而利用 LCA 或樹上 DP 來處理複雜的圖上路徑詢問。
24
25 */
26
27 struct BCC {
28     int n, timer;
29     vector<vector<int>>> adj;
30     vector<int> dfn, low;
31     vector<bool> is_cut; // 紀錄點是否為割點
32     vector<vector<int>>> bcc_nodes; // 儲存每個 BCC 內的點集
33     stack<int> st;
34
35     void init(int _n) {
36         n = _n;
37         timer = 0;
38         adj.assign(n + 1, vector<int>());
39         dfn.assign(n + 1, 0);
40         low.assign(n + 1, 0);
41         is_cut.assign(n + 1, false);
42         bcc_nodes.clear();
43         while (!st.empty()) st.pop();
44     }
45
46     void add_edge(int u, int v) {
47         adj[u].push_back(v);
48         adj[v].push_back(u);
49     }
50
51     void dfs(int u, int p = -1) {
52         dfn[u] = low[u] = ++timer;
53         st.push(u);
54         int children = 0; // 紀錄 DFS 樹上的子節點數量
55
56         for (int v : adj[u]) {
57             if (v == p) continue; // 不走回頭路 (父節點)
58
59             if (dfn[v]) {
60                 // v 已經拜訪過，代表這是一條 Back-edge (回邊)
61                 low[u] = min(low[u], dfn[v]);
62             } else {
63                 // v 沒拜訪過，代表這是一條 Tree-edge (樹邊)
64                 children++;
65                 dfs(v, u);
66
67                 // 子節點回來後，用子節點的 low 更新自己的 low
68                 low[u] = min(low[u], low[v]);
69
70                 // 判斷割點與收集 BCC 的核心條件
71                 if (low[v] >= dfn[u]) {
72                     is_cut[u] = true;
73                     vector<int> comp;
74
75                     // 從 stack 把屬於這個 BCC 的點全部彈出
76                     while (true) {
77                         int w = st.top();
78                         st.pop();
79                         comp.push_back(w);
80                         if (w == v) break; // 注意：退到 v 就停下
81                     }
82                     // 割點 u 會同時屬於多個 BCC，把它加進去但【不要】從 stack
83                     彈出它
84                     comp.push_back(u);
85                     bcc_nodes.push_back(comp);
86                 }
87             }
88         }
89
90         // 根節點特判：DFS 起點如果有小於 2 個子樹，則不是割點
91         if (p == -1 && children < 2) {
92             is_cut[u] = false;
93         }
94     }
95
96     void solve() {
97         for (int i = 1; i <= n; ++i) {
98             if (!dfn[i]) {
99                 dfs(i);
100             }
101         }
102
103         // 處理極端情況：圖中若存在孤立的單個點，將其視為一個 BCC

```

```

94         if (bcc_nodes.empty() && n == 1) {
95             bcc_nodes.push_back({1});
96         }
97     }
98 }
99 }
100 };

```

5.8 匈牙利演算法

用法：無權二分圖最大匹配，透過回溯尋找增廣路徑，以達到最大匹配。
分為左 (S)、右 (T) 兩側，以及 left 陣列紀錄右側所選的對象

時間複雜度： $O(V \cdot E)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 30005; // 依題目需求調整右側/左側最大點數
5 int n, m, e;           // n: 左側點數, m: 右側點數
6 vector<int> G[MAXN];    // G[u] 存左側點 u 連向右側的哪些點
7 int lef[MAXN];         // lef[v] 記錄右側點 v 配對到的左側點
8 bool T[MAXN];          // T[v] 記錄右側點 v 在單次 DFS 中是否已拜訪
9
10 bool match(int u) { // 核心 DFS: 幫左側點 u 尋找增廣路徑
11     for (int v : G[u]) {
12         if (T[v]) continue;
13         T[v] = true;
14
15         // 若右側點 v 還沒選對象，或其原本對象可以讓出位子
16         if (lef[v] == -1 || match(lef[v])) {
17             lef[v] = u; // 配對成功
18             return true;
19         }
20     }
21     return false;
22 }
23
24 int solve() { // 執行最大匹配
25     int ans = 0;
26     memset(left, -1, sizeof(left)); // 依照需求假設點編號為 0-based 或 1-based
27
28     // 假設左側點編號為 1 ~ n (若為 0 ~ n-1 請自行改迴圈範圍)
29     for (int i = 1; i <= n; ++i) {
30         memset(T, false, sizeof(T));
31         if (match(i)) {
32             ans++;
33         }
34     }
35     return ans;
36 }
37
38 int main() {
39     while (cin >> n >> m) {
40         for (int i = 0; i <= n; ++i) {
41             G[i].clear();
42         }
43
44         int u, v; // 讀取每個邊，存到 adjacency list 中
45         for (int i = 0; i < e; ++i) {
46             cin >> u >> v;
47             G[u].push_back(v);
48         }
49
50         // 依照需求做更改，這裡的範例是：找到的最大匹配數等於邊的數量，即完成最大匹配
51         if (solve() == e) cout << "YES\n";
52         else cout << "NO\n";
53     }
54     return 0;
55 }

```

5.9 KM 演算法

用法：適用於左右節點數皆為 N 的帶權完全二分圖

時間複雜度： $O(N^3)$

```

1 /*
2 前提：完美匹配與虛擬節點
3 KM 演算法要求左右兩側的節點數必須相等（形成完全二分圖）
4 如果在題目中左右節點數不同，或者某些點之間沒有連線，我們必須加入「虛擬節點」與「權重為 0（或極小值）的虛擬邊」來補齊，才能讓 KM 演算法順利運作。
5 */
6
7 struct KM {
8     int n;
9     vector<vector<long long>> weight; // weight[u][v] 表示左 u 到右 v 的權重
10    vector<long long> lx, ly, slack; // lx, ly 為左右點的「期望值（頂標）」，slack 為鬆弛量
11    vector<int> match_y;           // match_y[v] = u 代表右側 v 匹配給左側 u
12    vector<int> pre;              // 紀錄右側節點的交替樹路徑，用於 BFS 回溯
13    vector<bool> vis_y;           // 紀錄右側節點是否在當前交替樹中
14    const long long INF = 1e18;
15
16    void init(int _n) {
17        n = _n;
18        // 為了方便，採用 1-based index，0 作為虛擬的空節點
19        weight.assign(n + 1, vector<long long>(n + 1, 0));
20        lx.assign(n + 1, 0);
21        ly.assign(n + 1, 0);
22        match_y.assign(n + 1, 0);
23    }
24
25    void add_edge(int u, int v, long long w) {
26        weight[u][v] = max(weight[u][v], w); // 若有多重邊，保留權重最大的
27    }
28
29    // BFS 尋找增廣路徑並更新頂標

```

```

30    void bfs(int root) {
31        slack.assign(n + 1, INF);
32        vis_y.assign(n + 1, false);
33        pre.assign(n + 1, 0);
34
35        int y = 0;
36        match_y[0] = root; // 將起始點放在虛擬節點 match_y[0]
37
38        while (true) {
39            int x = match_y[y]; // 當前考慮的左側節點
40            long long delta = INF;
41            int next_y = 0;
42            vis_y[y] = true;
43
44            // 遍歷所有右側節點，計算 slack 量
45            for (int i = 1; i <= n; ++i) {
46                if (!vis_y[i]) {
47                    long long diff = lx[x] + ly[i] - weight[x][i];
48                    if (diff < slack[i]) {
49                        slack[i] = diff;
50                        pre[i] = y; // 紀錄從哪個右側節點轉移過來
51                    }
52                    if (slack[i] < delta) {
53                        delta = slack[i];
54                        next_y = i;
55                    }
56                }
57            }
58
59            // 更新交替樹中所有點的頂標（期望值）
60            for (int i = 0; i <= n; ++i) {
61                if (vis_y[i]) {
62                    lx[match_y[i]] -= delta;
63                    ly[i] += delta;
64                } else {
65                    slack[i] -= delta;
66                }
67            }
68
69            y = next_y; // 走到下一個右側節點
70            if (match_y[y] == 0) break; // 找到未匹配的右側節點，增廣路徑結束
71        }
72
73        // 回溯並更新匹配狀態
74        while (y != 0) {
75            match_y[ly] = match_y[pre[y]];
76            y = pre[y];
77        }
78    }
79
80    long long solve() {
81        // 初始化左側頂標為連接邊的最大權重
82        for (int i = 1; i <= n; ++i) {
83            lx[i] = -INF;
84            for (int j = 1; j <= n; ++j) {
85                lx[i] = max(lx[i], weight[i][j]);
86            }
87        }
88
89        // 對每一個左側節點進行匹配
90        for (int i = 1; i <= n; ++i) {
91            bfs(i);
92        }
93
94        // 計算最大權重總和
95        long long max_weight = 0;
96        for (int i = 1; i <= n; ++i) {
97            if (match_y[i] != 0) {
98                max_weight += weight[match_y[i]][i];
99            }
100        }
101        return max_weight;
102    }
103 };

```

6 幾何

6.1 點與向量基礎 (Point & Vector)

用法：幾何基礎，定義 struct 與 operator。包含浮點數誤差處理 sign，以及核心工具 dot (內積) 與 cross (外積)。

時間複雜度： $O(1)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double EPS = 1e-9; // 用來處理浮點數誤差的極小值
5
6 // 符號判斷函數（極度重要！）
7 // 判斷一個浮點數是 正數 (1)、負數 (-1) 還是 0(0)
8 int sign(double x) {
9     if (fabs(x) < EPS) return 0;
10    return x < 0 ? -1 : 1;
11 }
12
13 struct Point {
14     double x, y; // 通用 double
15     Point(double _x = 0, double _y = 0) : x(_x), y(_y) {}
16
17     // 向量加減法與常數乘除
18     Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
19     Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
20     Point operator*(double d) const { return Point(x * d, y * d); }
21     Point operator/(double d) const { return Point(x / d, y / d); }
22
23     // 判斷兩點是否相等（利用 sign 來避免浮點數誤差）
24     bool operator==(const Point& p) const {
25         return sign(x - p.x) == 0 && sign(y - p.y) == 0;
26     }

```

```

27
28     bool operator<(const Point &p) const{
29         if(x != p.x) return x < p.x;
30         return y < p.y;
31     }
32 };
33
34 using Vector = Point; // 在幾何中，向量和點的表示法一樣，給它一個別名增加可讀性
35
36 // 1. 內積 (Dot Product)，計算方式：x1*x2 + y1*y2
37 // 實戰意義：用來判斷兩向量夾角。大於 0 為銳角，小於 0 為鈍角，等於 0 為垂直。
38 double dot(Vector a, Vector b) {
39     return a.x * b.x + a.y * b.y;
40 }
41
42 // 2. 外積 (Cross Product)，計算方式：x1*y2 - x2*y1
43 // 實戰意義：用來判斷方向與面積。
44 // 若 cross(a, b) > 0，代表 b 在 a 的逆時針方向（偏左）
45 // 若 cross(a, b) < 0，代表 b 在 a 的順時針方向（偏右）
46 // 若 cross(a, b) == 0，代表 a 和 b 平行或共線
47 double cross(Vector a, Vector b) {
48     return a.x * b.y - a.y * b.x;
49 }

```

6.2 判斷線段、矩形相交 (Segment Intersection)

用法：利用外積進行跨立實驗。避開斜率除以零的問題，可處理端點相切與共線重疊。

時間複雜度： $\mathcal{O}(1)$

```

1 #include <algorithm>
2 using namespace std;
3
4 // 依賴前一節的 Point 結構與 sign, cross 函數
5
6 // 輔助函數：判斷點 P 是否在線段 AB 的「邊界框 (Bounding Box)」內
7 // 實戰意義：當三點共線時，單靠外積無法確認 P 是否在線段上（可能在延長線上）
8 bool on_segment(Point p, Point a, Point b) {
9     return p.x >= min(a.x, b.x) && p.x <= max(a.x, b.x) &&
10         p.y >= min(a.y, b.y) && p.y <= max(a.y, b.y);
11 }
12
13 // 核心函數：判斷線段 AB 與線段 CD 是否相交
14 bool segment_intersect(Point a, Point b, Point c, Point d) {
15     // 1. 計算四個外積值（跨立實驗）
16     // 意義：判斷 C 點和 D 點分別在直線 AB 的哪一側
17     double c1 = cross(b - a, c - a);
18     double c2 = cross(b - a, d - a);
19
20     // 意義：判斷 A 點和 B 點分別在直線 CD 的哪一側
21     double c3 = cross(d - c, a - c);
22     double c4 = cross(d - c, b - c);
23
24     // 2. 規範相交 (Strictly Intersect)
25     // 如果 C, D 在 AB 兩側（外積正負號相反，相乘 < 0）
26     // 且 A, B 在 CD 兩側，則保證兩線段相交
27     if (sign(c1) * sign(c2) < 0 && sign(c3) * sign(c4) < 0) {
28         return true;
29     }
30
31     // 3. 特殊情況處理（端點相交 或 共線重疊）
32     // 如果外積為 0，代表該點在另一條直線上。
33     // 接著必須用 on_segment 確保它真的落在「線段區間」內，而不是在延長線上。
34     if (sign(c1) == 0 && on_segment(c, a, b)) return true;
35     if (sign(c2) == 0 && on_segment(d, a, b)) return true;
36     if (sign(c3) == 0 && on_segment(a, c, d)) return true;
37     if (sign(c4) == 0 && on_segment(b, c, d)) return true;
38
39     return false; // 以上皆非，則不相交
40 }

```

6.3 點與線段關係 (Point & Line)

用法：判斷 1. 點是否在線段上 2. 求點到直線/線段最短距離。利用內外積避免解方程式與浮點數除以零。

時間複雜度： $\mathcal{O}(1)$

```

1 #include <cmath>
2 using namespace std;
3
4 // 依賴前面的 Point, Vector, sign, cross, dot
5
6 // 輔助函數：求向量長度
7 // 原理：向量自己跟自己做內積，等於長度的平方
8 double length(Vector a) {
9     return sqrt(dot(a, a));
10 }
11
12 // 1. 判斷點 P 是否在線段 AB 上
13 // 原理：
14 // (1) cross(p-a, b-a) == 0 代表三點共線
15 // (2) dot(a-p, b-p) <= 0 代表 PA 向量與 PB 向量方向相反（即 P 夾在 A, B 中間）
16 bool on_segment(Point p, Point a, Point b) {
17     return sign(cross(p - a, b - a)) == 0 && sign(dot(a - p, b - p)) <= 0;
18 }
19
20 // 2. 點 P 到「直線」AB 的距離
21 // 原理：平行四邊形面積 = 底 * 高 => 高 = 面積 / 底
22 double dist_to_line(Point p, Point a, Point b) {
23     // 面積剛好是外積的絕對值，底邊長是 AB 的長度
24     return fabs(cross(b - a, p - a)) / length(b - a);
25 }
26
27 // 3. 點 P 到「線段」AB 的最短距離（賽場極易錯考點）
28 // 原理：用內積判斷 P 點的「投影」是否掉出線段外
29 double dist_to_segment(Point p, Point a, Point b) {
30     if (a == b) return length(p - a); // A, B 根本是同一個點
31
32     // 鈍角測試 1：角 PAB 是鈍角，代表 P 在 A 的外側，離 A 最近

```

```

33     if (sign(dot(b - a, p - a)) < 0) return length(p - a);
34
35     // 鈍角測試 2：角 PBA 是鈍角，代表 P 在 B 的外側，離 B 最近
36     if (sign(dot(a - b, p - b)) < 0) return length(p - b);
37
38     // 如果都不是鈍角，代表 P 的投影乖乖落在線段 AB 上，距離就是點到直線距離
39     return dist_to_line(p, a, b);
40 }

```

6.4 任意多邊形面積

用法：利用外積求多邊形面積（鞋帶公式）。頂點必須依順時針或逆時針順序給定。支援凹凸多邊形。

時間複雜度： $\mathcal{O}(N)$

```

1 #include <cmath>
2 using namespace std;
3
4 // 依賴前面的 Point 結構與 cross 函數
5
6 const int MAXV = 1005;
7 Point poly[MAXV]; // 儲存多邊形的頂點（必須依照順時針或逆時針順序排列）
8 int n; // 頂點數量
9
10 // 求任意多邊形面積 (Shoelace Formula)
11 double polygon_area() {
12     double area = 0;
13     for (int i = 0; i < n; ++i) {
14         // 讓最後一個點與第 0 個點連線，形成封閉圖形
15         int next = (i + 1) % n;
16         area += cross(poly[i], poly[next]);
17     }
18     // 外積和有可能是負的（取決於順逆時針），所以取絕對值並除以 2
19     return fabs(area) / 2.0;
20 }
21
22 /*
23 補充：假設兩個矩形的左下角與右上角座標分別為
24 矩形 A：左下 (x1, y1)，右上 (x2, y2)
25 矩形 B：左下 (x3, y3)，右上 (x4, y4)
26
27 把矩形投影到 X 軸上，就變成了兩個線段 [x1, x2] 與 [x3, x4]
28 它們重疊的長度，等於「兩個右邊界的較小值」減去「兩個左邊界的較大值」
29 如果算出來是負數，代表它們根本沒碰到，直接跟 0 取最大值即可
30
31 因此計算矩形交集面積即為
32 int W = max(0, min(x2, x4) - max(x1, x3));
33 int H = max(0, min(y2, y4) - max(y1, y3));
34 int intersect_area = W * H;
35 */

```

6.5 凸包 Convex Hull

用法：求包圍所有點的最小凸多邊形。先排 X 再排 Y 。由左至右建下半部，由右至左建上半部。利用外積判斷並踢除右轉點。

時間複雜度： $\mathcal{O}(N \log N)$

```

1 #include <algorithm>
2 using namespace std;
3
4 // 依賴前面的 Point 結構與 cross 函數
5
6 const int MAXN = 100005;
7 Point pts[MAXN]; // 存放所有輸入的點
8 Point hull[MAXN]; // 存放最終構成凸包的點
9 int n; // 輸入點的數量
10
11 // 回傳凸包上的頂點數量
12 int convex_hull() {
13     // 點太少直接就是凸包
14     if (n <= 2) {
15         for (int i = 0; i < n; ++i) hull[i] = pts[i];
16         return n;
17     }
18
19     // 1. 將所有點由左至右、下至上排序
20     sort(pts, pts + n);
21
22     int m = 0; // m 代表目前凸包 (hull) 裡面有幾個點
23
24     // 2. 蓋下半部凸包 (Lower Hull) - 從左走到右
25     for (int i = 0; i < n; ++i) {
26         // 【核心邏輯】
27         // cross <= 0 代表「往右轉」或是「直走」。
28         // 凸包的邊緣必須一直「往左轉」，如果發現往右轉（凹進去了），就把最後一個點
29         // 踢掉！
30         // (若想保留共線的點，把 <= 改成 < 即可)
31         while (m >= 2 && cross(hull[m - 1] - hull[m - 2], pts[i] - hull[m - 2]) <= 0) {
32             m--; // 踢掉最後一個點
33         }
34         hull[m++] = pts[i]; // 把新點加進來
35     }
36
37     // 3. 蓋上半部凸包 (Upper Hull) - 從右走回左
38     // 注意：下半部已經蓋了 m 個點，為了不踢掉下半部的點，我們設定底線為 t
39     int t = m + 1;
40     for (int i = n - 2; i >= 0; --i) {
41         // 邏輯跟上面一模一樣，只是倒著走
42         while (m >= t && cross(hull[m - 1] - hull[m - 2], pts[i] - hull[m - 2]) <= 0) {
43             m--;
44         }
45         hull[m++] = pts[i];
46     }
47
48     // 4. 起點（最左下的點）會在頭尾被重複加入一次，所以總數量要減 1
49     return m - 1;
50     // 然後有了這些點又可以面積公式直接得到整個的面積
51 }

```


7 樹論

7.1 Disjoint Set

用法: 解決集合、快速合併的問題。優化必須路徑壓縮，可選小掛大時間複雜度: $\mathcal{O}(1)$

```
1 struct DSU {
2     vector<int> f, sz; // 避免使用 size 撞名
3
4     void init(n){
5         f.resize(n);
6         sz.assign(n, 1); // 將每個獨立集合的大小初始化為 1
7         for (int i = 0; i < n; ++i) f[i] = i;
8     }
9
10    int find(int x) { // 路徑壓縮
11        return f[x] == x ? x : (f[x] = find(f[x]));
12    }
13
14    bool isSameSet(int a, int b) {
15        return find(a) == find(b);
16    }
17
18    // 回傳 bool 有助於在 Kruskal 演算法中判斷是否成功加上一條邊，否則 void 即可
19    bool unite(int a, int b) {
20        int fa = find(a), fb = find(b);
21        if (fa == fb) return false;
22
23        // 小掛大優化：永遠讓 fa 是比較大的集合
24        if (sz[fa] < sz[fb]) swap(fa, fb);
25        sz[fa] += sz[fb];
26        f[fb] = fa;
27        return true;
28    }
29 };
```

7.2 Kruskal 最小生成樹

用法: 需搭配並查集使用。將所有邊排序後，利用併查集檢查是否形成環。

時間複雜度: $\mathcal{O}(E \log E)$

```
1 // 讓 Edge 支援比較大小，排序時會自動依據權重 w 由小到大排
2 struct Edge {
3     int u, v;
4     long long w;
5     bool operator<(const Edge& rhs) const {
6         return w < rhs.w;
7     }
8 };
9
10 struct Kruskal {
11     int n;
12     vector<Edge> edges;
13
14     void init(int _n) {
15         n = _n;
16         edges.clear();
17     }
18
19     void add_edge(int u, int v, long long w) {
20         edges.push_back({u, v, w});
21     }
22
23     // 回傳最小生成樹的總權重。若圖不連通則回傳 -1
24     long long solve() {
25         sort(edges.begin(), edges.end()); // 1. 將所有邊依權重排序
26
27         DSU dsu;
28         dsu.init(n + 1); // 2. 呼叫併查集 (預設 1-based index)
29
30         long long total_weight = 0;
31         int edge_cnt = 0; // 記錄成功連接了幾條邊
32
33         for (auto& e : edges) {
34             // 3. unite 右回傳 true，代表 u 和 v 原本不相連，合併成功 (無環)
35             if (dsu.unite(e.u, e.v)) {
36                 total_weight += e.w;
37                 edge_cnt++;
38
39                 // V 個點只要 V-1 條邊即完全連通，提早結束優化
40                 if (edge_cnt == n - 1) break;
41             }
42         }
43
44         // 檢查是否有成功把所有點連通
45         return (edge_cnt == n - 1) ? total_weight : -1;
46     }
47 };
48
49 /*
50 延伸題型
51 找第二小的 Minimum Spanning Tree
52 改成 Maximum Spanning Tree
53 對 Min ST 做 DFS
54 */
```

8 DP: 分錢幣問題, Kadane's algo 找最大子序列問題

DP 通用解題四步驟:

1. 定義陣列意義：明確寫出 $dp[i]$ (或 $dp[i][j]$) 代表什麼。
2. 找出遞迴關係式：推導出當前狀態如何由過去狀態轉移而來。

3. 設定邊界條件：初始化起點或基礎 dp 值 (如: $dp[0] = 0$)。
4. 確認遍歷順序：決定迴圈是從前到後 (Bottom-Up) 還是後到前 (Top-Down) 計算。(通常是 Bottom-Up，思考”我是如何到這一步的”去推)

8.1 01 Knapsack 背包

用法: 解決物品不可拆分的情況，若可拆分用 Greedy 拿比例最佳的部分時間複雜度: $\mathcal{O}(N)$

```
1 /*
2 題目敘述：給定 N 個物品的重量 Wi 和價值 Vi，和個容量為 W 的背包。選取若干件物品
3 放入背包，在不超過背包容量的情況下，背包內物品價值總和最大為何？
4 定義 dp[i][j] 表示：在考慮前 i 個物品，且考慮重量 <=j 的情況下，所能得到的最大價值
5 是多少 (假設 index 0 不放東西)
6 初始：對於所有的 i=0, dp[i][j] = 0 (不考慮任何物品的情況下，最大價值是 0)，也可以
7 直接預設所有位置都是 0
8 轉移函式: dp[i][j] = max(dp[i-1][j], dp[i-1][j-Wi] + Vi);
9 -> 兩種情況:
10 1. 不放第 i 件物品，最大價值與前一項相同
11 2. 放入第 i 件物品，最大價值為 扣除目標重量後的最大價值 加上當前物品的價值之總和
12 適用情況：需要 backtrack
13 -> 從 dp[0][0] 開始檢查:
14 如果 dp[i][j] == dp[i-1][j] 表示沒有拿第 i 個，接著讓 i = i-1 繼續跑
15 否則我們有拿第 i 個，接著跳到 dp[i-1][j-w[i]]
16 重複直到 i=0 (沒東西了) 或 j=0 (重量用完了)
17 */
18 int dp[1000][1000]; // 初始化為 0
19 memset(dp, 0, sizeof(dp)); // 初始化為 0
20 for (int i = 1; i <= MAXN; ++i) // 從考慮第一個物品到第 N 個物品
21 {
22     for (int j = 0; j <= W; ++j) // 如果當前考慮重量小於 Wi，代表沒辦法裝下第
23         i 個物品，直接繼承前一項
24         dp[i][j] = dp[i-1][j];
25     for (int j = W; j <= MAXW; ++j) // 否則就用 transition function 找 max
26         dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i]] + v[i]);
27 }
28 cout << dp[1000][1000] << '\n'; // 最終答案在最右下那格
29
30 /*
31 ===== 滾動陣列優化 =====
32 概念：在計算考慮第 i 個物品時，我們只會用到第 i-1 行的資料，可以看成陣列中的上一
33 筆舊資料
34 透過想法上的極致壓縮，覆蓋無用資料空間，可將空間複雜度降至  $\mathcal{O}(W)$ 
35 限制是 j 必須由大往小跑，如果由小至大會重複使用到陣列資料導致答案錯誤
36 */
37 int dp[1000];
38 memset(dp, 0, sizeof(dp));
39 for (int i = 1; i <= MAXN; ++i) // 注意一樣是要跑 MAXN 次，我們只是把陣列大小減
40     少了
41     {
42         for (int j = MAXW; j >= w[i]; --j) // dp[j] 在等號左邊表示當前，等號右邊表
43             示上一筆資料
44             dp[j] = max(dp[j], dp[j-w[i]] + v[i]); // 不需要考慮 j < w[i] 的情況，
45             因為它會繼承上一個結果
46     }
47 cout << dp[1000] << '\n';
48
49 /*
50 *** 恰好裝滿問題 / 子集和 ***
51 題目敘述：給定 N 樣物品，每件物品的重量分別為 Wi，問可不可以將物品分成等重的兩堆。
52
53 題解：首先判斷總重是否為二的倍數，如果不是就直接輸出 NO 然後下一個 case
54 如果可以的話就使用一維 DP 解，總重/2 當作 MAXW (dp 大小)，每件行李的重量是價值，
55 看是否存在 dp[MAXW] = MAXW (在裝入一半行李的情況下，總重剛好也是一半)
56
57 多背包共用商品 ===
58 題目：N 個物品 (同一項商品可被多個人同時取用)，G 個人各自有容量為 Wi 的背包，求所
59 有人能拿的最大價值總和。
60
61 題解：每個人挑選互相獨立，不需重複計算。(查表)
62 1. 以所有商品跑一次標準一維 0/1 背包，每次讀入 Wi 與 Vi 時就 run 一次，建立 dp
63 陣列。
64 2. 結束後跑一遍 dp[i] = max(dp[i], dp[i-1]) 確保單調性。
65 3. 讀入每個人的容量 W，總答案 ans += dp[W]。
66 */
```

8.2 LCS 最長共同子序列 (Sequence)

用法: 二維 DP 與 Backtrack 找原始子序列

時間複雜度: $\mathcal{O}(NM)$

```
1 // LCS (線性 DP): dp[i][j] 代表的是 s1 的前 i 個字元，與 s2 的前 j 個字元。這是一個從索引 0 開始的前綴
2
3 struct LCS {
4     // 求解 LCS 並回傳最長共同子序列的字串
5     // 若只需長度，直接回傳 dp[n][m] 即可
6     string solve(const string& s1, const string& s2) {
7         int n = s1.length(), m = s2.length();
8
9         // dp[i][j] 代表 s1 的前 i 個字元與 s2 的前 j 個字元的最長共同子序列長度
10        // 宣告 n+1 與 m+1 是為了讓 index 0 作為「空字串」的邊界條件 (全為 0)
11        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
12
13        // 1. 狀態轉移 (填表)
14        for (int i = 1; i <= n; ++i) {
15            for (int j = 1; j <= m; ++j) {
16                if (s1[i-1] == s2[j-1]) {
```

```

17         // 若字元相同，繼承左上角的答案 + 1
18         dp[i][j] = dp[i - 1][j - 1] + 1;
19     } else {
20         // 若字元不同，取上方或左方的最大值
21         dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
22     }
23 }
24 }
25
26 // 2. 回溯 (Backtracking) 找回原本的字串
27 string res = "";
28 int i = n, j = m;
29 while (i > 0 && j > 0) {
30     if (s1[i - 1] == s2[j - 1]) {
31         // 如果字元相同，這一定是 LCS 的一部分
32         res += s1[i - 1];
33         i--; j--; // 往左上角回退
34     } else if (dp[i - 1][j] > dp[i][j - 1]) {
35         // 如果不同，往數值比較大的方向走 (此處為上方)
36         i--;
37     } else {
38         // 否則往左方走
39         j--;
40     }
41 }
42
43 // 因為是從尾巴推回去的，最後要把字串反轉
44 reverse(res.begin(), res.end());
45 return res;
46 }
47 };
48
49 /*
50 補充: Longest Palindrome Sequence 最長迴文子序列
51 給定一個字串，問你該字串的最長迴文子序列 "長度" 為多少
52 做法就是對 s1 與 s2=reverse(s1) 做一次 LCS，即可得到 "最長長度"
53
54 變化 2: 求最少插入/刪除次數以形成迴文
55 如果只求長度 -> strlen - LPS(s)
56 */

```

8.3 區間 DP 概念 (最長迴文子序列)

用法: 區間 DP 是由字串的內部向外擴張，利用已經算好的較短子字串，來推導出較長字字串的答案。

時間複雜度: $O(N^2)$

```

1 // LPS (區間 DP) : dp[i][j] 代表的是同一個字串 s 裡面，從索引 i 到 j 的這段閉區
2 間 (也就是子字串 s[i...j])
3
4 struct IntervalDP {
5     // 求解最長迴文子序列 (LPS) 長度
6     int solveLPS(const string& s) {
7         int n = s.length();
8         if (n == 0) return 0;
9
10        // dp[i][j] 代表字串 s 區間 [i, j] 內的最長迴文子序列長度
11        vector<vector<int>> dp(n, vector<int>(n, 0));
12
13        // 1. 狀態轉移 (填表)
14        // 注意 i 是從字串的「尾巴」往「頭」掃
15        for (int i = n - 1; i >= 0; --i) {
16            // Base case: 單一字元本身就是長度為 1 的迴文
17            dp[i][i] = 1;
18
19            // j 從 i 的下一個位置開始向右擴展
20            for (int j = i + 1; j < n; ++j) {
21                if (s[i] == s[j]) {
22                    // 若頭尾字元相同，長度為內部區間的迴文長度 + 2
23                    // 因為 i 是倒著掃，j 是正著掃，所以 dp[i+1][j-1] 絕對已經算過了
24                    dp[i][j] = dp[i + 1][j - 1] + 2;
25                } else {
26                    // 若頭尾不同，看是捨棄左邊 (s[i]) 還是捨棄右邊 (s[j]) 比較長
27                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
28                }
29            }
30        }
31
32        // 最終答案就是涵蓋整個字串的區間 [0, n-1]
33        return dp[0][n - 1];
34    }
35
36    // 求解變成迴文的最少編輯距離 (包含插入、刪除、替換)
37    // 如果求最少 插入/刪除 字元，才能讓字串變成迴文的題目，請用 strlen - LPS(s)
38    pair<int, string> solvePalindromicDistance(const string& s) {
39        // 1. 找最少編輯次數
40        int n = s.length();
41        if (n == 0) return {0, ""};
42
43        // dp[i][j] 代表將字串 s 區間 [i, j] 變成迴文的最少編輯次數
44        vector<vector<int>> dp(n, vector<int>(n, 0));
45
46        // i 從字串的「尾巴」往「頭」掃
47        for (int i = n - 1; i >= 0; --i) {
48            dp[i][i] = 0; // 單一字元本身就是迴文，編輯距離為 0
49
50            // j 從 i 的下一個位置開始向右擴展
51            for (int j = i + 1; j < n; ++j) {
52                if (s[i] == s[j]) {
53                    // 頭尾相同，不需要編輯
54                    // 當 j == i+1 時，dp[i+1][j-1] 即 dp[i+1][i] 會是 0 (因為預設值)
55                    dp[i][j] = dp[i + 1][j - 1];
56                } else {
57                    // 頭尾不同，取三種操作的最小值 + 1
58                    // 1. dp[i+1][j] -> 插入/刪除對應 s[i]
59                    // 2. dp[i][j-1] -> 插入/刪除對應 s[j]

```

```

59        // 3. dp[i+1][j-1] -> 直接把 s[i] 替換成 s[j] 或反之
60        dp[i][j] = min({dp[i + 1][j], dp[i][j - 1], dp[i + 1][j - 1]}) + 1; // 如果有定義 cost 要記得改
61    }
62 }
63
64 // 2. 狀態回溯 (Backtracking)
65 string left_part = "", right_part = "";
66 int i = 0, j = n - 1;
67
68 while (i < j) {
69     if (s[i] == s[j]) { // 若頭尾相同，直接加入答案
70         left_part += s[i];
71         right_part += s[j];
72         i++; j--;
73     } else {
74         // 若頭尾不同，檢查是哪條最佳路徑轉移過來的
75         bool can_sub = (dp[i + 1][j - 1] + 1 == dp[i][j]); // 選擇用替換的
76         bool can_del_i = (dp[i + 1][j] + 1 == dp[i][j]); // 選擇刪除 i 的
77         bool can_del_j = (dp[i][j - 1] + 1 == dp[i][j]); // 選擇刪除 j 的
78
79         char best_c = 127; // 用來尋找字典序最小的字元 (ASCII 上限)
80         int choice = -1; // 1: 替換, 2: 處理左側, 3: 處理右側
81
82         // 評估替換策略 (選擇兩者中較小的字元)
83         if (can_sub) {
84             char c = min(s[i], s[j]);
85             if (c < best_c) { best_c = c; choice = 1; }
86         }
87         // 評估處理左側策略 (對應 s[i])
88         if (can_del_i) {
89             char c = s[i];
90             if (c < best_c) { best_c = c; choice = 2; }
91         }
92         // 評估處理右側策略 (對應 s[j])
93         if (can_del_j) {
94             char c = s[j];
95             if (c < best_c) { best_c = c; choice = 3; }
96         }
97     }
98
99     // 寫入最佳字元
100    left_part += best_c;
101    right_part += best_c;
102
103    // 根據最佳策略移動指標
104    if (choice == 1) { i++; j--; }
105    else if (choice == 2) { i++; }
106    else if (choice == 3) { j--; }
107 }
108
109 // 若最後指標交會於同一個字元，該字元放正中間
110 if (i == j) {
111     left_part += s[i];
112 }
113
114 // 組合最終字串 (右半部需要反轉)
115 reverse(right_part.begin(), right_part.end());
116
117 return {dp[0][n-1], left_part + right_part};
118 }
119 };
120
121 /*
122 有關迴文的其他題目補充
123 變化 1: 計算迴文子序列的「總數」 (排容原理)
124 if (s[i] != s[j]) dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]
125 if (s[i] == s[j]) dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1
126
127 變化 2: 跨字串 (左 A 右 B) 的迴文
128 令 dp[i][j] 表示「A 從索引 i 開始的後綴」與「B 從索引 j 開始的前綴」所能構成的最長條件迴文長度。
129 if (A[i] == B[j]) dp[i][j] = dp[i+1][j-1] + 2
130 if (A[i] != B[j]) dp[i][j] = max(dp[i+1][j], dp[i][j-1])
131
132 */

```

8.4 跳格子

用法: 給定一組陣列，固定跳躍距離，問你最少需要跳幾次，以及所跳過的格子 index。如果數值達到 long long，INF 需設為 1e18 或 0x3f3f3f3f3f3f3f3f

時間複雜度: $O(N \times K)$

```

1 //
2 題目出處: PUPC 2025 PC River Crossing
3 題目敘述: 給定一組陣列，1 表示可達，0 表示不可達，問你最少需要跳幾次可以跳到終點，以及跳時經過的路徑
4 如果不需要輸出路徑，可以直接用 greedy，但要輸出路徑就需要用 DP
5
6 #include <bits/stdc++.h>
7 using namespace std;
8
9 int main(){
10     int kase;
11     cin >> kase;
12     while(kase--){
13         int n;
14         cin >> n;
15         vector<int> v(n), ans(n, 0x3f3f3f3f), pre(n, -1);
16         // v 存原始陣列，ans 存抵達該格的最短步數，pre 存 在最短步數的情況下，我上一格走的位置
17         for(int i=0; i<n; ++i) cin >> v[i]; // 讀入陣列
18         ans[0] = 0; // 初始化: 跳到第 0 格的距離是 0
19         for(int i=1; i<n; ++i){ // 接下來每次遍例我的前三格

```

```

21     if(v[i] == 0) continue; // 如果該格不可達，
22     for(int j=1; j<=3; ++j){ // 這裡的 3 是題目設定的，請改成該題目設置的數值
23         if(i-j >= 0 && v[i-j] == 1 && ans[i-j] != 0x3f3f3f3f){
24 // 判斷：1. 在邊界內， 2. 該格可走， 3. 重複確認該格可走（如不可走，ans[i-j] 會
直接 skip，保留 INF）
25         if(ans[i] >= ans[i-j] + 1){ // 注意題目要求：相同距離的情況下，格子要
選哪一個，在這題中紀錄的格子要越小越好
26             ans[i] = ans[i-j] + 1;
27             pre[i] = i-j;
28         }
29     }
30 }
31 }
32 if(ans[n-1] == 0x3f3f3f3f) cout << -1 << '\n';
33 else{
34     cout << ans[n-1] << '\n';
35 }
36 int i = n-1;
37 stack<int> st;
38 st.push(n-1);
39 while(i > 0){
40     st.push(pre[i]);
41     i = pre[i];
42 }
43
44     bool first = true;
45     while(!st.empty()){
46         if(!first) cout << ' ';
47         first = false;
48         cout << st.top();
49         st.pop();
50     }
51     cout << '\n';
52 }
53 }
54 return 0;
55 }

```

8.5 LIS / LDS 最長遞增/遞減子序列

用法：嚴格遞增用 `lower_bound`，非嚴格改用 `upper_bound`。替換策略下的 `dp` 陣列長度正確，但內容不一定是真實子序列。

時間複雜度： $\mathcal{O}(N \log N)$

```

1 二維套疊問題 (Russian Doll Envelopes / 箱子堆疊)
2 題意：給你一堆信封，每個有寬度和高度，問你最多可以把幾個信封像俄羅斯娃娃一樣套在
一起
3 解法：將寬度由小到大排序；當寬度相同時，高度必須由大到小排。接著，直接對高度的陣
列求 LIS
4
5 橋樑/連線不交叉問題 (Building Bridges)
6 題意：南岸和北岸各有 N 座城市，題目給定連線配對。要求建最多座橋，且橋與橋之間不能
交叉
7 解法：把南岸的城市編號由小到大排序，然後對它們對應的北岸城市編號求 LIS
8
9 最少覆蓋路徑 / 導彈攔截系統 (Dilworth 定理)
10 題意：系統每次只能攔截「高度遞減」的飛彈。給你一連串飛彈的高度，問你「最少」需要裝
備幾套攔截系統？
11 解法：「最需要的遞減子序列數量」等價於「最長遞增子序列 (LIS) 的長度」。求出 LIS
長度就直接是答案！
12
13 最長雙坡子序列 (Bitonic Subsequence)
14 題意：求一個數列，先嚴格遞增再嚴格遞減的最長長度。
15 解法：從左到右跑一次 LIS，記錄每個位置結尾的 LIS 長度；再從右到左跑一次 LIS（相當
於 LDS），記錄每個位置開頭的 LDS 長度。兩者相加減 1 的最大值即為所求。
16
17 */
18
19 // 1. 最長遞增子序列 (LIS) - (嚴格) 遞增
20 int get_lis(const vector<int>& arr) {
21     vector<int> dp;
22     for (int x : arr) {
23         // lower_bound: 找第一個 >= x 的位置（若要求非嚴格遞增，改用
upper_bound）
24         auto it = lower_bound(dp.begin(), dp.end(), x);
25         if (it == dp.end()) {
26             dp.push_back(x); // 比所有數字都大，序列長度 + 1
27         } else {
28             *it = x; // 貪心策略：替換成較小的結尾，增加後續連接的潛力
29         }
30     }
31     return dp.size(); // dp 陣列的長度就是 LIS 的長度（注意：dp 內容不一定是真
實 LIS）
32 }
33
34 // 2. 最長遞減子序列 (LDS) - (嚴格) 遞減
35 int get_lds(const vector<int>& arr) {
36     vector<int> dp;
37     for (int x : arr) {
38         // LDS 只需要搭配 greater<int>() 來找第一個 <= x 的位置
39         // 若要求非嚴格遞減，改用 less_equal<int>() 等自訂比較
40         auto it = lower_bound(dp.begin(), dp.end(), x, greater<int>());
41         if (it == dp.end()) {
42             dp.push_back(x);
43         } else {
44             *it = x;
45         }
46     }
47     return dp.size();
48 }

```

9 Greedy

9.1 一維區間全覆蓋

用法：需要先把輸入轉化成一個包含 `l, r` 的 `struct`，自訂 `operator<`，起始位置由小到大排序

時間複雜度： $\mathcal{O}(N \log N)$

```

1 /*
2 題目敘述：給定 N 個灑水器，每個灑水器包含座標 P 與半徑 R，已知草坪長度為 L，寬度
為 W，請問最少需要開啟幾個灑水器就能覆蓋整個區間（或指定哪些灑水器）
3
4 題解：先將灑水器圓轉化為一個 struct point，用 l, r 區間表示能覆蓋的左右範圍，然
後自訂 operator<
5 1. 用開始位置排序
6 2. 每次都在 小於當前右界的區間中，選擇能往右最多的那個
7 3. 重複直到右區間 <L，得到最少數量；或者是在當前右邊界內找不到可選擇的表示不存在
解
8 */
9 const double eps = 1e-8;
10 struct point{
11     double L, R;
12     bool operator<(const point&o) const{
13         return L < o.L;
14     }
15 };
16
17 vector<point> v;
18 double pos, rad;
19 for(int i=0; i<n; ++i){
20     cin >> pos >> rad;
21     double real;
22     if(rad - w/2.0 < eps) continue; // 判斷如果半徑小於寬度就不可能被選，直接
不加入
23     real = sqrt(rad*rad - (w/2.0)*(w/2.0)); // real 表示實際可覆蓋距離
24     v.push_back({pos-real, pos+real});
25 }
26 sort(v.begin(), v.end());
27
28 int cnt = 0; // 總計數，如果要指定哪些灑水器就用 vector 存每次加入的
29 int i = 0;
30 double furthest = 0.0;
31 bool noAns = false;
32
33 while(L - furthest > eps){ // L > furthest
34     double cur = furthest;
35     while(i < v.size() && v[i].L - furthest < eps){
36         cur = max(cur, v[i].R);
37         ++i;
38     }
39     if(cur - furthest < eps){
40         noAns = true;
41         break;
42     }
43     furthest = cur;
44     ++cnt;
45 }
46 if(noAns) cout << -1 << '\n';
47 else cout << cnt << '\n';

```

9.2 間格跳躍

用法：給定一組陣列，距離為 `index` 的值，問你最少需要跳多少次才能到終點

時間複雜度： $\mathcal{O}(N)$

```

1 int jump(vector<int>& nums) {
2     int jumps = 0; // 總跳躍次數
3     int current_end = 0; // 「當前這一步」能到達的最遠邊界
4     int farthest = 0; // 探索過程中，發現「下一步」能到達的最遠距離
5     bool noAns = false;
6
7     // 注意：迴圈只走到 nums.size() - 2
8     // 因為如果已經站在最後一個位置，就不需要再跳了
9     for (int i = 0; i < nums.size() - 1; ++i) {
10         // 邊走邊看，更新下一跳的最遠潛力
11         farthest = max(farthest, i + nums[i]);
12
13         // 當走到了「當前這步」的極限邊界時，就必須結算，進行「下一跳」
14         if (i == current_end) {
15             if (current_end == farthest){ // 如果被迫跳但發現一步都沒跳就代表走
不到終點
16                 noAns = true;
17                 break;
18             }
19             jumps++;
20             current_end = farthest; // 將邊界擴展到剛才探索到的最遠距離
21
22             if (current_end >= nums.size()-1) break; // 提前結束的優化
23         }
24     }
25     if (noAns) return -1;
26     return jumps;
27 }

```

9.3 最大不重疊區間

用法：對一組任務做排程，請問最多可以移除多少個任務 -> 最早結束的先選。

時間複雜度： $\mathcal{O}(N \log N)$

```

1 int countRemoves(vector<pair<int, int>> &intervals) {
2     if(intervals.empty()) return 0; // 不需要移除任何東西
3
4     // 用結束時間做排序
5     sort(intervals.begin(), intervals.end(), [](const auto&a, const
auto&b){return a.second < b.second;});
6
7     int removed = 0;
8     int cur_end = intervals[0].second; // 一開始必定選第一個任務
9     for(int i=1; i<intervals.size(); ++i){ // 遍歷所有任務
10         if(cur_end > intervals[i].first) ++removed; // 如果我目前的 end 能覆蓋
到當前 i 的左邊界就移除 i
11         else cur_end = intervals[i].second; // 否則我就要選 i，並設為當前右邊
界

```

```

12     }
13
14     return removed;
15 }

```

9.4 最小覆蓋區間

用法: 給定一組陣列, 包含開始時間與結束時間, 求最少需要幾個房間就能完成所有任務

時間複雜度: $\mathcal{O}(N \log N)$

```

1 struct Interval {
2     int st, ed;
3     bool operator<(const Interval &o) const {
4         if (st != o.st) return st < o.st; // 先以開始時間排序
5         return ed < o.ed; // 開始時間相同, 較早結束的排前面
6     }
7 };
8
9 int minMeetingRooms(vector<Interval>& intervals) {
10     if (intervals.empty()) return 0;
11
12     // 1. 依照開始時間排序
13     sort(intervals.begin(), intervals.end());
14
15     // 2. 建立一個 Min-Heap, 專門用來儲存「使用中會議室的結束時間」
16     // !! 如果需要印出順序, 只要把 int(結束時間) 改成 pair<int, int>(結束時間, 房間編號)!!
17     priority_queue<int, vector<int>, greater<int>> min_heap;
18
19     // 3. 掃描每一個會議
20     for (const auto& interval : intervals) {
21         // 如果 pq 不為空, 且最早結束的會議已經可以使用了 (當前會議的開始時間 >= 最早空出來的房間時間)
22         if (!min_heap.empty() && interval.st >= min_heap.top()) {
23             min_heap.pop(); // 該會議室空出來了, 把舊的結束時間 pop 掉 (準備重複利用)
24         }
25
26         // 把當前會議的結束時間塞進去 (代表佔用了一間會議室)
27         // 不管是開了一間新房間, 還是接續前一間, 當前的結束時間都要記錄
28         min_heap.push(interval.ed);
29     }
30
31     return min_heap.size(); // 最終 Heap 裡面剩下幾個元素, 就代表我們同時需要幾間會議室
32 }

```

9.5 區間選點問題

用法: 將區間依照右界由小到大排序。遍歷所有區段, 如果當前區間的左界大於目前選定點的右界, 代表必須多選一個點

時間複雜度: $\mathcal{O}(N \log N)$

```

1 struct Interval {
2     int l, r;
3     // 依右界由小到大排序, 右界相同時左界小的優先
4     bool operator<(const Interval& o) const {
5         if (r != o.r) return r < o.r;
6         return l < o.l;
7     }
8 };
9
10 int minPoints(vector<Interval>& intervals) {
11     if (intervals.empty()) return 0;
12     sort(intervals.begin(), intervals.end());
13
14     int cnt = 1;
15     int current_r = intervals[0].r;
16
17     for (int i = 1; i < intervals.size(); ++i) {
18         // 若當前區間左界 > 目前紀錄的右界, 代表無法共用同一個點
19         if (intervals[i].l > current_r) {
20             cnt++;
21             current_r = intervals[i].r; // 更新右界
22         }
23     }
24     return cnt;
25 }

```

9.6 最小化最大延遲問題

用法: 按照到期時間從早到晚處理。記錄當下累積的耗時, 並與該任務的死線相減, 取最大的延遲時間

時間複雜度: $\mathcal{O}(N \log N)$

```

1 struct Job {
2     int time, deadline;
3     // 依到期時間由早到晚排序
4     bool operator<(const Job& o) const {
5         return deadline < o.deadline;
6     }
7 };
8
9 int minimizeLateness(vector<Job>& jobs) {
10     sort(jobs.begin(), jobs.end());
11
12     int current_time = 0;
13     int max_lateness = 0;
14
15     for (const auto& job : jobs) {
16         current_time += job.time;
17         // 延遲時間 = 完成時間 - 到期時間。若提早完成則延遲為 0
18         int lateness = max(0, current_time - job.deadline);
19         max_lateness = max(max_lateness, lateness);
20     }
21     return max_lateness;
22 }

```

9.7 Huffman 樹

用法: 每次合併兩堆, 成本是兩堆的重量和, 求最小總成本

時間複雜度: $\mathcal{O}(N \log N)$

```

1 long long huffmanCost(const vector<int>& weights) {
2     // 宣告 Min-Heap
3     priority_queue<long long, vector<long long>, greater<long long>> pq;
4     for (int w : weights) pq.push(w);
5
6     long long total_cost = 0;
7
8     // 每次挑選最小的兩堆合併, 直到只剩一堆
9     while (pq.size() > 1) {
10         long long first = pq.top(); pq.pop();
11         long long second = pq.top(); pq.pop();
12
13         long long merged = first + second;
14         total_cost += merged;
15         pq.push(merged); // 將合併後的新重量塞回 Heap
16     }
17
18     return total_cost;
19 }

```

9.8 任務調度問題

用法: 依照懲罰由大到小排序, 每項工作依序嘗試可不可以放在 $D_i - T_i + 1, D_i - T_i, \dots, 1, 0$, 如果有空間就放進去, 否則延後執行。

時間複雜度: $\mathcal{O}(N)$

```

1 struct Task {
2     int deadline, penalty;
3     // 依懲罰由大到小排序
4     bool operator<(const Task& o) const {
5         return penalty > o.penalty;
6     }
7 };
8
9 // 利用並查集快速找尋「最晚可用的空間天數」
10 struct DSU {
11     vector<int> parent;
12     DSU(int n) {
13         parent.resize(n + 1);
14         for (int i = 0; i <= n; ++i) parent[i] = i;
15     }
16     int find(int x) {
17         return parent[x] == x ? x : (parent[x] = find(parent[x]));
18     }
19 };
20
21 long long taskScheduling(vector<Task>& tasks) {
22     sort(tasks.begin(), tasks.end());
23
24     int max_deadline = 0;
25     for (const auto& t : tasks) {
26         max_deadline = max(max_deadline, t.deadline);
27     }
28
29     DSU dsu(max_deadline);
30     long long total_penalty = 0;
31
32     for (const auto& t : tasks) {
33         // 尋找此任務到期日之前, 最晚可用的空檔
34         int available_slot = dsu.find(t.deadline);
35
36         if (available_slot > 0) {
37             // 有空位: 佔用這天, 並將這天的 parent 指向前一天
38             dsu.parent[available_slot] = dsu.find(available_slot - 1);
39         } else {
40             // 沒空位: 只能放棄並承受懲罰
41             total_penalty += t.penalty;
42         }
43     }
44     return total_penalty;
45 }

```

9.9 多機調度問題

用法: 將工作依耗時「由大到小」排序。每次將工作交給最快空閒出來的機器。

時間複雜度: $\mathcal{O}(N \log N)$

```

1 struct Machine {
2     int id;
3     long long completion_time;
4     // Min-Heap: 優先選擇最快空閒的機器。若時間相同, 優先分給 ID 小的
5     bool operator>(const Machine& o) const {
6         if (completion_time != o.completion_time)
7             return completion_time > o.completion_time;
8         return id > o.id;
9     }
10 };
11
12 long long multiprocessorScheduling(vector<int>& jobs, int m) {
13     // 工作依耗時由大到小排序 (Longest Processing Time first)
14     sort(jobs.begin(), jobs.end(), greater<int>());
15
16     priority_queue<Machine, vector<Machine>, greater<Machine>> pq;
17     for (int i = 0; i < m; ++i) pq.push({i, 0});
18
19     // 依序把工作派給最快結束的機器
20     for (int job_time : jobs) {
21         Machine earliest = pq.top();
22         pq.pop();
23
24         earliest.completion_time += job_time;
25         pq.push(earliest);
26     }
27 }

```



```

28 // 找出所有機器中最晚完工的時間
29 long long max_time = 0;
30 while (!pq.empty()) {
31     max_time = max(max_time, pq.top().completion_time);
32     pq.pop();
33 }
34
35 return max_time;
36 }

```

10 MISC

10.1 模板

用法: 競賽用基礎模板

時間複雜度: $\mathcal{O}(\text{HowFastYouType})$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 int main(){
6     ios::sync_with_stdio(0); cin.tie(0); // 或使用 scanf
7     return 0;
8 }

```

10.2 一些概念

用法: 想不到怎麼解或優化的時候可以參考這些想法

時間複雜度:

```

1 /*
2 前綴和
3 枚舉 (dfs、減少變數降低維度、集合拆半後枚舉、剪枝)
4 貪心 (每次都選局部最佳, 但不能保證局部最佳解是全域最佳解)
5 雙指針
6 滑動窗口
7 Random Select(QuickSort)
8 對答案二分搜
9 */

```

10.3 C++ 函式庫

用法: 好用的東西可以參考

時間複雜度: *NULL*

```

1 __builtin_popcount(x): 回傳整數 x 轉為二進位後 1 的數量 (若處理 long long 需使用)
2 __builtin_popcountll(x):
3 __builtin_clz(x): 回傳二進位前導 0 的個數 (Count Leading Zeros)
4 gcd(a, b), lcm(a, b): 求最大公因數 (C++17 引入, 需引入 <numeric>, 舊版編譯器可使用內建的 __gcd(a, b))
5 string 相關: stoi(s) 將字串轉為 int;
6 to_string(num) 將數字轉為字串;
7 s.substr(pos, len) 擷取子字串
8
9 #include <algorithm>
10
11 sort(bg, ed) sort(bg, ed, cmp): 預設由小到大排序
12
13 min(a, b) min(list): 取最小值
14 max(a, b) max(list): 取最大值
15
16 min_element(first, last): 找尋連續資料中最小元素
17 max_element(first, last): 找尋連續資料中最大元素
18
19 find(first, last, val): 尋找元素
20 lower_bound(first, last, val): 尋找第一個 ">=" x 的元素"位置", 如果不存在, 則回傳 last
21 upper_bound(first, last, val): 尋找第一個 ">" x 的元素"位置", 如果不存在, 則回傳 last
22
23 next_permutation(first, last): 將序列順序轉換成下一個字典序, 如果存在回傳 true, 反之回傳 false
24 prev_permutation(first, last): 將序列順序轉換成上一個字典序, 如果存在回傳 true, 反之回傳 false
25
26 unique(bg, ed): 將相鄰重複元素移至尾端, 回傳新陣列無重複部分尾端的 iterator (或指標)
27 reverse(bg, ed): 反轉區間內的元素順序
28
29 #include <cctype>
30
31 isalnum(int c): 判斷是否為數字或英文
32 isalpha(int c): 判斷是否為英文
33 isdigit(int c): 判斷是否為數字
34
35 islower(int c): 判斷是否為小寫字母
36 isupper(int c): 判斷是否為大寫字母
37
38 tolower(int c): 將字母轉成小寫字母
39 toupper(int c): 將字母轉成大寫字母
40
41 #include <stack>
42
43 s.push(T a): 插入頂端元素, 複雜度  $\mathcal{O}(1)$ 
44 s.pop(): 刪除頂端元素, 複雜度  $\mathcal{O}(1)$ 
45 s.top(): 回傳頂端元素, 複雜度  $\mathcal{O}(1)$ 
46 s.size(): 回傳元素個數, 複雜度  $\mathcal{O}(1)$ 
47 s.empty(): 回傳是否為空, 複雜度  $\mathcal{O}(1)$ 
48
49 #include <queue>
50
51 q.push(T a): 插入尾端元素, 複雜度  $\mathcal{O}(1)$ 
52 q.pop(): 刪除頂端元素, 複雜度  $\mathcal{O}(1)$ 
53 q.front(): 回傳頂端元素, 複雜度  $\mathcal{O}(1)$ 
54 q.size(): 回傳元素個數, 複雜度  $\mathcal{O}(1)$ 

```

```

55 q.empty(): 回傳是否為空, 複雜度  $\mathcal{O}(1)$ 
56
57 pq.push(T a): 插入元素 a, 複雜度  $\mathcal{O}(\log \text{size})$ 
58 pq.pop(): 刪除頂端元素, 複雜度  $\mathcal{O}(\log \text{size})$ 
59 pq.top(): 回傳頂端元素, 複雜度  $\mathcal{O}(1)$ 
60 pq.size(): 回傳元素個數, 複雜度  $\mathcal{O}(1)$ 
61 pq.empty(): 回傳是否為空, 複雜度  $\mathcal{O}(1)$ 
62
63 #include <deque>
64
65 dq.push_front(T a), dq.push_back(T a): 插入頂端/尾端元素, 複雜度  $\mathcal{O}(1)$ 
66 dq.pop_front(), dq.pop_back(): 刪除頂端/尾端元素, 複雜度  $\mathcal{O}(1)$ 
67 dq.front(), dq.back(): 回傳頂端/尾端元素, 複雜度  $\mathcal{O}(1)$ 
68 dq.size(): 回傳元素個數, 複雜度  $\mathcal{O}(1)$ 
69 dq.empty(): 回傳是否為空, 複雜度  $\mathcal{O}(1)$ 
70
71 #include <set>
72
73 s.size(): 回傳元素個數, 複雜度  $\mathcal{O}(1)$ 
74 s.empty(): 回傳是否為空, 複雜度  $\mathcal{O}(1)$ 
75 s.clear(): 清除元素, 複雜度  $\mathcal{O}(\text{size})$ 
76 s.insert(T1 a): 加入元素 a, 複雜度  $\mathcal{O}(\log \text{size})$ 
77 s.erase(iterator first, iterator last): 刪除 [first, last), 若沒有指定 last 則只刪除 first
78 s.erase(T1 a): 刪除鍵值 a, 複雜度  $\mathcal{O}(\log \text{size})$ 
79 s.find(T1 a): 回傳指向鍵值 a 的迭代器, 若不存在則回傳 s.end(), 複雜度  $\mathcal{O}(\log \text{size})$ 
80 s.count(T a): 計算有幾個元素 a
81 s.lower_bound(T1 a): 回傳指向第一個鍵值大於等於 a 的迭代器, 複雜度  $\mathcal{O}(\log \text{size})$ 
82 s.upper_bound(T1 a): 回傳指向第一個鍵值大於 a 的迭代器, 複雜度  $\mathcal{O}(\log \text{size})$ 
83
84 #include <map>
85
86 m.size()
87 m.empty()
88 m.clear()
89 m.count()
90 m.erase(iterator first, iterator last)
91 m.erase(T1 a)
92 m.find(T1 a)
93 m.lower_bound(T1 a)
94 m.upper_bound(T1 a)

```